AUTONOMOUS EVOLUTIONARY AGENT SWARM (v2.7)

Printable Desk Reference — Product Discovery & Planning (Phase A), Goal-Loop Outcome Engine & Swarm Loop (Phase B)

`/product-discovery` Phase A Master (Scout & Plan)

`/invoke-swarm` Phase B Master (Until Goal Met)

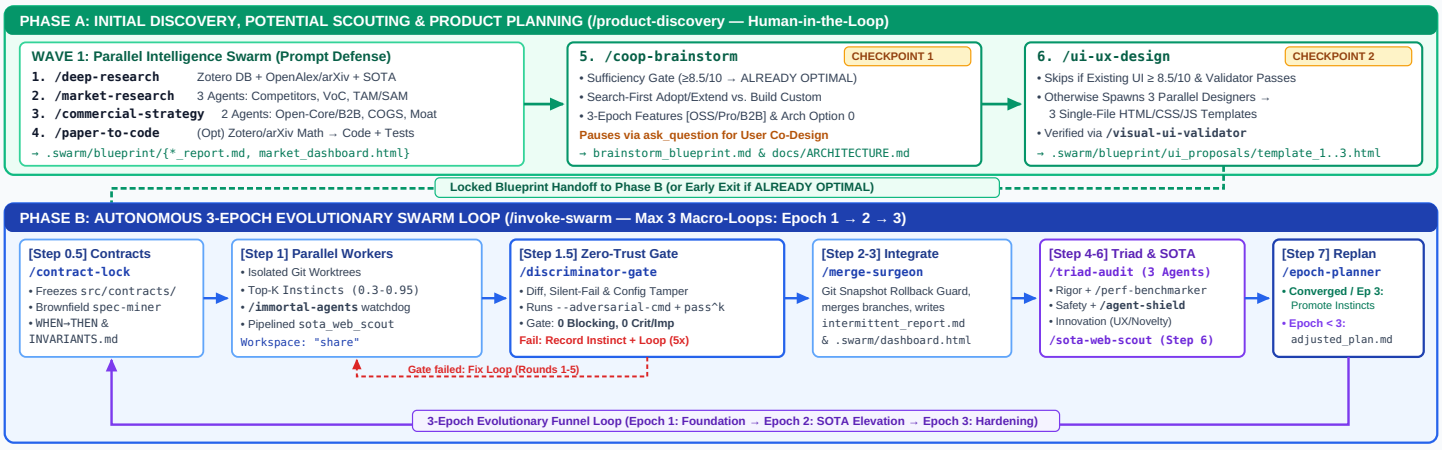
1. QUICK DECISION GUIDE — "WHICH SKILL DO I TYPE RIGHT NOW?"

Pick by your immediate goal; every skill works standalone or via /product-discovery & /invoke-swarm

Starting a brand-new product/idea (Full Phase A)?	`/product-discovery`	Ready to build & evolve code autonomously (3 Epochs)?	`/invoke-swarm`
Backend / CLI / SDK product plan (skip HTML UI mockups)?	`/product-discovery no-ui`	Just 1 implementation wave + merge (no 3-epoch loop)?	`/invoke-swarm single-epoch`
Search my Zotero PDFs + high-impact papers + web SOTA?	`/deep-research`	Freeze shared types + mine brownfield invariants?	`/contract-lock`
Competitors, GitHub OSS parity, VoC pain & TAM/SAM/SOM?	`/market-research`	Zero-trust code review, silent-failure & pass^k gate?	`/discriminator-gate`
Open-Core vs. B2B Enterprise licensing, COGS & kill-risks?	`/commercial-strategy`	Run Rigor, Safety & Innovation audit on existing repo?	`/triad-audit`
Co-brainstorm features, Search-First Adopt & architecture?	`/coop-brainstorm`	Scan secrets, prompt-injection, Unicode & MCP tax?	`/agent-shield`
Generate 3 interactive single-file HTML/CSS/JS UI mockups?	`/ui-ux-design`	Turn paper math/equations into verified code + unit tests?	`/paper-to-code`

2. END-TO-END ARCHITECTURE & EXECUTION FLOW (PHASE A → PHASE B)

Phase A (/product-discovery) is on-demand only; Phase B (/invoke-swarm) runs Steps 0.5 → 7 autonomously



3. PHASE A SKILLS — INITIAL DISCOVERY, OPPORTUNITY SCOUTING, OPEN-CORE/B2B & PRODUCT PLANNING

Run all at once via /product-discovery or invoke any individual skill standalone on any repository

SKILL & CALLING MODES	WHEN TO USE (TRIGGER SCENARIO)	SUB-AGENTS & CORE CAPABILITIES	PERSISTENT OUTPUTS
`/product-discovery` `default` `no-ui` `tech-only` `market-only`	**New product OR existing repo audit.** Crawls web/literature/market potentials & plans product (2 waves, 2 checkpoints).	**Wave 1:** `/deep-research` + `/market-research` + `/commercial-strategy`. **Checkpoint 1:** `/coop-brainstorm` (can declare `ALREADY OPTIMAL` if repo ≥ 8.5/10). **Wave 2:** `/ui-ux-design`.	All `.swarm/blueprint/*` files + `docs/ARCHITECTURE.md`
`/deep-research`	**Literature & Zotero grounding.** Academic rigor + your own curated Zotero notes/PDFs + web SOTA.	**3 Parallel Streams (Prompt Defense):** (A) Personal Zotero DB via `zotero_tool.py` (collections, notes, PDF grep), (B) High-impact papers (OpenAlex citations, arXiv, PubMed), (C) Web/OSS Build-vs-Adopt crawl.	`.swarm/blueprint/deep_research_report.md` `.swarm/sota/sota_knowledge_base.md`
`/market-research`	**Competitive & buyer validation.** Greenfield ideas OR existing repos (Truth-in-Code parity mode).	**Spawns 3 Agents:** `competitor_teardown_scout` (incumbents, SaaS, OSS velocity/licenses), `voice_of_customer_miner` (HN/Reddit/GitHub Switch Triggers), `tam_icp_gtm_analyst` (bottom-up TAM/SAM/SOM & ICP).	`.swarm/blueprint/market_research_report.md` `.swarm/blueprint/market_dashboard.html`
`/commercial-strategy`	**Open-Core & B2B monetization.** Designing OSS vs. paid boundaries, pricing, COGS margins, and moats.	**Spawns 2 Agents:** `pricing_monetization_strategist` (MIT/Apache Core + `ee/` vs. BSL/AGPL, 3-tier OSS/Pro/B2B Enterprise gates, COGS) & `moat_and_risk_skeptic` (cannibalization, IP/GPL scan, kill-risks).	`.swarm/blueprint/commercial_strategy_report.md` `.swarm/blueprint/market_dashboard.html`
`/coop-brainstorm`	**Feature, arch & Search-First gate.** Interactive ideation + 1-10 Sufficiency Scorecard before coding.	**Sufficiency Gate (≥8.5/10 →** `ALREADY OPTIMAL` **) + 3 Pillars:** (1) Innovations (Net-ROI ≥ 8.0), (2) Search-First `[ADOPT]` / `[EXTEND]` / `[BUILD]` + `[OSS]` / `[Pro]` / `[B2B]`, (3) Arch Option 0 vs. Alternatives.	`.swarm/blueprint/brainstorm_blueprint.md` `docs/ARCHITECTURE.md`
`/ui-ux-design`	**Visual prototyping & UX selection.** Audits existing UI (≥8.5/10 skips regen) or builds 3 HTML mockups.	Audits existing UI via `web_ui_validator.py` (keeps if ≥ 8.5/10); otherwise proposes 3 distinct UI/UX philosophies, spawns 3 parallel UI Designers for self-contained HTML/CSS/JS files, and validates.	`.swarm/blueprint/ui_proposals/template_1..3.html` `.swarm/blueprint/ui_ux_proposals.md`
`/paper-to-code`	**Translating papers to verified math code.** Turning Zotero/arXiv equations into tested reference modules.	Extracts formal equations/algorithms from Zotero PDFs or arXiv, cross-verifies closed-form identities via SymPy (optionally a local math model), and writes golden unit tests.	`.swarm/paper_spec.md` + Reference Module & Unit Tests

4. PHASE B SKILLS — AUTONOMOUS 3-EPOCH SWARM LOOP & STANDALONE STAGE SKILLS

Run the full loop via /invoke-swarm or invoke any individual stage skill on any branch/repo

SKILL & CALLING MODES	WHEN TO USE (TRIGGER SCENARIO)	WHAT IT DOES & GATE ENFORCEMENT RULES	PERSISTENT OUTPUTS
`/invoke-swarm` `default (3-Epoch)` `single-epoch` `discriminate-only` `audit-only` `sota-only` `replan-only`	**Autonomous multi-agent build & evolution.** Executes Steps 0.5 → 7 across up to 3 Macro-Loops until convergence.	Locks contracts + invariants, spawns parallel worktree workers (injected with Top-K `instincts.json`), supervises via `immortal_watcher`, gates each worker with `discriminator`, merges, runs Triad + AgentShield + SOTA Scout, and replans.	`.swarm/swarm_tasks.json` `.swarm/dashboard.html` `.swarm/epoch_1..3/*`
`/contract-lock` `Step 0.5` `spec-miner`	**Before parallel coding.** Prevents signature collisions & mines brownfield invariants.	Defines and freezes shared cross-module interfaces (`src/contracts/`) on `main`. On brownfield repos, runs Sample & Expand `spec-miner` (15-file cap) to lock `WHEN→THEN` & `ALWAYS TRUE` invariants.	`src/contracts/*` `docs/INVARIANTS.md`
`/discriminator-gate` `Step 1.5` `pass^k`	**Zero-trust branch review.** Auditing a sub-agent or feature branch before allowing merge.	Inspects `git diff`, runs tests + `--adversarial-cmd` (with `--pass-hat-k`), Silent-Failure Hunter & Config/Contract Tampering check. Enforces a **categorical gate** (0 `BLOCKING_FAIL`, spec `PASS`, 0 Critical/Important; loops up to 5x).	`.swarm/instincts.json` `.swarm/lessons_learned.md`
`/merge-surgeon` `Step 3` `snapshots`	**Integrating parallel branches.** Merging multiple Git worktrees cleanly into `main`.	Creates pre-merge Git snapshots (`--snapshot-create`), merges approved worker branches (`git merge --no-ff`), resolves semantic conflicts honoring `src/contracts/`, and auto-restores (`--snapshot-restore`) on failure.	Integrated `main` branch + tag `swarm-epoch-<N>`
`/triad-audit` `default (All 3)` `rigor-only` `safety-only` `innovation-only`	**Deep codebase audit (Steps 4-5).** Finding math bugs, security holes, or UX/innovation gaps.	Spawns 3 parallel auditors: `rigor_auditor` (math/algo complexity, `pass^k` & `/perf-benchmarker`), `safety_auditor` (runs `/agent-shield` + threat tests), `innovation_auditor` (UX/visuals & methods). Scores 1-10.	`.swarm/epoch_<N>/audit_rigor.md` `audit_safety.md`, `audit_innovation.md`
`/agent-shield` `Step 4.2` `standalone`	**Security, Prompt-Injection & MCP audit.** Scanning secrets, Unicode, hooks & MCP tax.	Runs `scripts/agent_shield.py` across 5 categories: (1) Secrets & tracked `.env`, (2) Prompt-Injection & Trojan-Source Unicode, (3) Unsafe hook pipes (`curl|sh`), (4) MCP Context-Tax (≤5 servers), (5) Silent failures.	`.swarm/security_shield_report.md` (Grade `A-F`, `0-100`)
`/perf-benchmarker` `Step 4.1` `pass_hat_k`	**Latency, RAM & flakiness tracking.** Detecting speed, memory, or `pass^k` regressions.	Runs `scripts/perf_tracker.py` across K iterations, measuring wall-clock latency (ms), peak RSS memory (KB), `pass_at_k` (≥1/K), and `pass_hat_k` (K/K non-flaky passes). Flags regressions > 15%.	`.swarm/benchmarks.json`
`/visual-ui-validator` `UI Gate` `standalone`	**Validating HTML/JS UIs.** Checking UI prototypes or web apps for broken JS or empty canvases.	Runs `scripts/web_ui_validator.py`: verifies HTML DOM structure, rejects `"Lorem ipsum"` placeholders, runs `node --check` on every inline `<script>`, and checks headless Chromium console/bounding boxes.	JSON/CLI pass-fail report + optional screenshots
`/sota-web-scout` `Step 6` `standalone`	**External SOTA benchmarking.** Comparing your repo against leading tools & papers.	Crawls web, GitHub, and literature in parallel (under Prompt Defense Baseline) for SOTA visualizations, architectures, and Build-vs-Adopt packages. Scores **Innovation vs. SOTA (1-10)** and **Implementation vs. SOTA (1-10)**.	`.swarm/sota/sota_knowledge_base.md` `.swarm/epoch_<N>/sota_evaluation.md`
`/epoch-planner` & `/immortal-agents`	**Milestone replanning (Steps 3/7) & sub-agent revival watchdog.**	`/epoch-planner`: Writes `intermittent_report.md`, renders `dashboard.html`, checks convergence & writes `adjusted_plan.md`. `/immortal-agents`: Inspects `transcript.jsonl` & revives stalled workers (up to 5x).	`intermittent_report.md` `adjusted_plan.md` `.swarm/dashboard.html`

5. SUPPORTING ENGINEERING, GPU MATH & LIVE 3D EDITOR MCP SKILLS

Specialized execution contracts, binary runtime smoke testing, local GPU math proofs & singleton 3D scenes

SKILL	WHEN TO USE	CAPABILITIES & KEY COMMANDS
`/goal-loop`	**Hierarchical Tasks, Briefs & Hard-Cap Gate.**	Runs `scripts/goal_loop.py`: `--preflight` conflict scan, `--task-brief` & `--review-package` bundlers, AST call-site wiring check, and Hard-Cap Two-Verdict recording.
`/swarm-diagnostics`	**Diagnose slow, looping, or stalled sessions.**	Runs `scripts/swarm_diagnostics.py` over `transcript.jsonl` & `.swarm/` across 7 dimensions with mandatory `path:line` / `step_index` citations.
`/quality-validator`	**Smoke-testing compiled apps/games.**	Runs `scripts/quality_validator.py ./build/app --timeout 10` enforcing fresh verification evidence before completion claims.
`/math-reasoning` & `/tmux-orchestrator`	**Optional local math model & headless CLI daemons.**	Optional extra: routes derivations to a local math model (falls back to SymPy); manages multi-pane Linux `tmux` runners.

6. TWO-VERDICT GATE, HARD CAPS & 5-ROUND ESCALATING LOOP

- **No Arbitrary Point Math — Deterministic Invariants + Semantic Gate:** `goal_loop.py` checks binary `BLOCKING_FAIL` invariants (test exit code, AST wiring, zero stubs) & bundles `BASE_SHA..HEAD` diffs.
- **Two-Verdict Discriminator:** (1) **Spec Compliance** (`PASS | FAIL | UNVERIFIABLE` + `Review Focus` proof) & (2) **Severity Buckets**: `Critical`, `Important`, `Minor (Deferred)`.
- **Strict Hard Caps (Override Arithmetic):**
  `Any BLOCKING_FAIL / Spec FAIL / Critical → Cap 6.0`
  `Any Important → Cap 7.0` `Minor → Deferred to Ledger (Never Blocks)`
- **5-Round Escalating Fix Loop & Scoped Re-Review (`FIX_BASE..HEAD`):** Rounds 1–3 resume same worker; **Rounds 4–5 spawn a fresh worker sub-agent**; **Round 5 Breaker** logs a typed `Ruling:` (`OVERRIDE` / `SPLIT_TASK` / `RE_ARCHITECT_CONTRACT` / `ESCALATE_TO_USER`) in `.swarm/ledger.md`; rulings never overturn a `BLOCKING_FAIL`.
- **UI Craft & Unanchored Critique:** `ui_direction_dealer.py` deals directions by Surface Mode + Design Dials; `web_ui_validator.py` checks `1440/768/390` + `reduced-motion`; Assessment A runs unanchored before B.

7. PERSISTENT REPOSITORY FILE BUS (`.SWARM/` & `DESIGN.MD`)

`DESIGN.md & .swarm/surfaces/` — 3-Tier Design System, Tokens & Surface Briefs
`.swarm/blueprint/` — Phase A Discovery, Market & UI Prototypes
  ├─ `deep_research_report.md`, `market_research_report.md` & `market_dashboard.html`
  ├─ `brainstorm_blueprint.md` (with `Review Focus`) & `docs/INVARIANTS.md`
  └─ `ui_proposals/template_1..3.html` (3-viewport verified HTML UIs)
`.swarm/` — Phase B Briefs, Diff Packages, Ledgers & Telemetry
  ├─ `goal_loop.json`, `briefs/<ID>-brief.md` & `reviews/<ID>-review.diff.md`
  ├─ `ledger.md` (Round 5 Rulings) & `deferred_minor_ledger.md` (Minor notes)
  ├─ `dashboard.html`, `security_shield_report.md` & `diagnostics_report.md`
  └─ `instincts.json`, `epoch_<1..K>/` & `sota/sota_knowledge_base.md`